

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 415 — Past Questions & Materials
by Renance

Course: CSC 415
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 6
Levels: 400

This study pack was compiled and restructured by Renance for CSC 415 students at Federal University of Petroleum Resources, Effurun. It brings together 6 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 6 — CSC 415 - 2023/2024 - First Semester - Compiler Construction 1

Level 400 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.


 DEPARTMENT OF COMPUTER SCIENCE
 COLLEGE OF SCIENCE
 FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN
 2023/2024 FIRST SEMESTER EXAMINATION

CSC415 [Compiler Construction 1]
INSTRUCTIONS

- Answer Four questions
- Time Allowed: 2Hrs 20mins
- Do not mix up questions. Start a new question on a fresh page.
- All extra sheets used should be signed by the invigilator

QUESTION ONE

- For each of the following regular expression, list six strings which are in its language:
 (i) $(a(b+c)^*)^*d$ (ii) $(a+b)^*(c+d)$ (iii) $(a^*b^*)^*$. 6mks
- With the help of a diagram, describe the various phases of a compiler and translate the statement: position = initial + rate * 80. 6mks
- If a compiler written in machine language for computer A and the same compiler written in a high-level language for the same machine A, which of these compilers will produce faster code and why? 6mks

QUESTION TWO

- Draw a flowchart for a compiler using words: WHILE, WHEN, FOR, END, ELSE. 7mks
- Parse the following expression using operator precedence table and provide the resulting parse tree: $a + b * (c + d) + f$. 7mks
- What are the things that are checked by the semantic analyzer? 4mks

QUESTION THREE

- Briefly explain the following useful compiler construction tool: (i) parser generator (ii) syntax-directed translation engines (iii) data-flow engines. 6mks
- There are several reasons for separating the analysis phase of compiling into lexical and parsing. What are they? 6mks
- With the aid of a diagram, briefly explain the role of the parser in a compiler model. 6mks

QUESTION FOUR

- Since C/C++ are free format, newline characters are ignored during lexical analysis (except to serve as white space delimiters and to count lines for diagnostic purposes). Name at least two high-level programming languages for which newline characters would not be ignored for syntax analysis. 4mks
- A non-deterministic finite automaton (NFA) is a mathematical model that consists of various items. What are they? 7mks
- Draw a transition graph for an NFA that recognizes the language $(a|b)^*abb$. Where the set of states are $\{0, 1, 2, 3\}$ and the input symbol alphabet is $\{a, b\}$. 7mks

Handwritten notes and diagrams:

- herical Syntax Semantics*
- Code gen*
- CG*
- CG*
- lowest to highest*
- avoids recursion of parse tree*
- avoids recursion of parse tree*
- Portability of compiler*
- code optimization*
- code for the parser*
- Simple design*
- Compiled*
- Interpreted*
- Python*
- 0-Stop States*
- Input symbols*
- Output symbols*
- Can a state have more than one loop*
- ab* a/b abb*
- a set of states designated as accepting*
- designating as accepting*
- a transition function / input symbol*
- State is reached is transitioned to*
- Page 1 of 2*

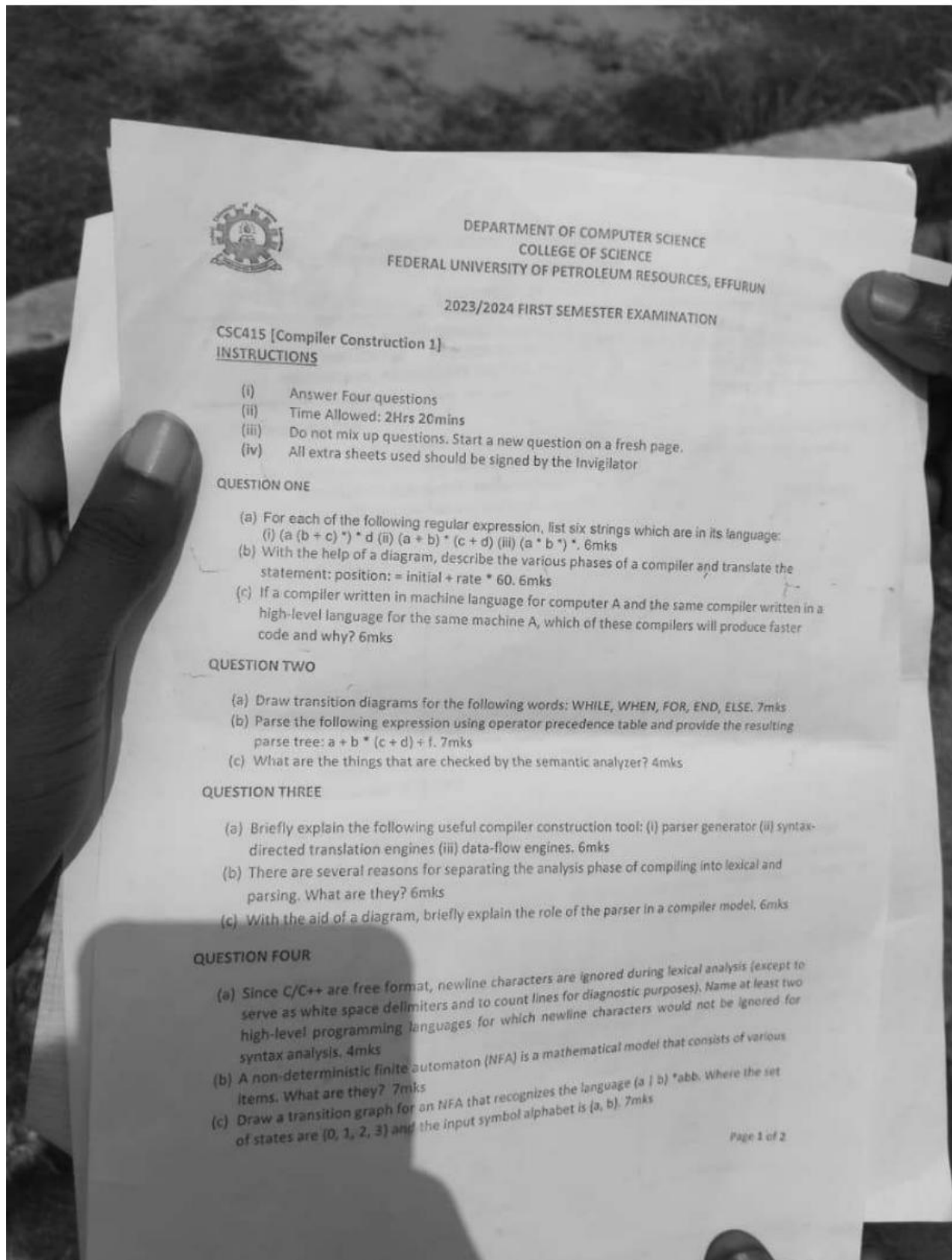
```

graph LR
    0((0)) -- a --> 1((1))
    1 -- b --> 2((2))
    2 -- b --> 3((3))
    style 0 fill:#fff,stroke:#000
    style 1 fill:#fff,stroke:#000
    style 2 fill:#fff,stroke:#000
    style 3 fill:#fff,stroke:#000
  
```

Paper 2 of 6 — CSC 415 - 2023/2024 - First Semester - Compiler Construction 1

Level 400 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



Paper 3 of 6 — CSC 415 - 2023/2024 - First Semester - Compiler Construction I

Level 400 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



 DEPARTMENT OF COMPUTER SCIENCE
 COLLEGE OF SCIENCE
 FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN
 2023/2024 FIRST SEMESTER EXAMINATION

CSC415 [Compiler Construction 1]
INSTRUCTIONS

(i) Answer Four questions
 (ii) Time Allowed: 2Hrs 20mins
 (iii) Do not mix up questions. Start a new question on a fresh page.
 (iv) All extra sheets used should be signed by the invigilator

QUESTION ONE

(a) For each of the following regular expression, list six strings which are in its language:
 (i) $(a(b+c)^*)^*d$ (ii) $(a+b)^*(c+d)$ (iii) $(a^*b^*)^*$. 6mks
 (b) With the help of a diagram, describe the various phases of a compiler and translate the statement: position = initial + rate * 80. 6mks
 (c) If a compiler written in machine language for computer A and the same compiler written in a high-level language for the same machine A, which of these compilers will produce faster code and why? 6mks

QUESTION TWO

(a) Draw a flowchart for a compiler using words: WHILE, WHEN, FOR, END, ELSE. 7mks
 (b) Parse the following expression using operator precedence table and provide the resulting parse tree: $a + b * (c + d) + f$. 7mks
 (c) What are the things that are checked by the semantic analyzer? 4mks

QUESTION THREE

(a) Briefly explain the following useful compiler construction tool: (i) parser generator (ii) syntax-directed translation engines (iii) data-flow engines. 6mks
 (b) There are several reasons for separating the analysis phase of compiling into lexical and parsing. What are they? 6mks
 (c) With the aid of a diagram, briefly explain the role of the parser in a compiler model. 6mks

QUESTION FOUR

(a) Since C/C++ are free format, newline characters are ignored during lexical analysis (except to serve as white space delimiters and to count lines for diagnostic purposes). Name at least two high-level programming languages for which newline characters would not be ignored for syntax analysis. 4mks
 (b) A non-deterministic finite automaton (NFA) is a mathematical model that consists of various items. What are they? 7mks
 (c) Draw a transition graph for an NFA that recognizes the language $(a|b)^*abb$. Where the set of states are {0, 1, 2, 3} and the input symbol alphabet is {a, b}. 7mks

Page 1 of 2

Can a state have more than one loop
 a set of states designated as accepting
 designating as accepting
 a transition function / input symbol

State is reached is transitioned to

(a|b)*abb

Python
 Perl
 C++
 Java
 Ruby
 Perl

Compiler
 Interpreter

Parser
 Semantic Analyzer
 Lexical Analyzer

Code generator
 lexical
 Syntax
 Semantics
 LCg
 COPT
 CG

lowest to highest

avoids recursion of parse tree
 generates parse tree
 Parses
 Portability
 of
 compiler

Simple design

Can a state have more than one loop
 a set of states designated as accepting
 designating as accepting
 a transition function / input symbol

State is reached is transitioned to

(a|b)*abb

Page 1 of 2

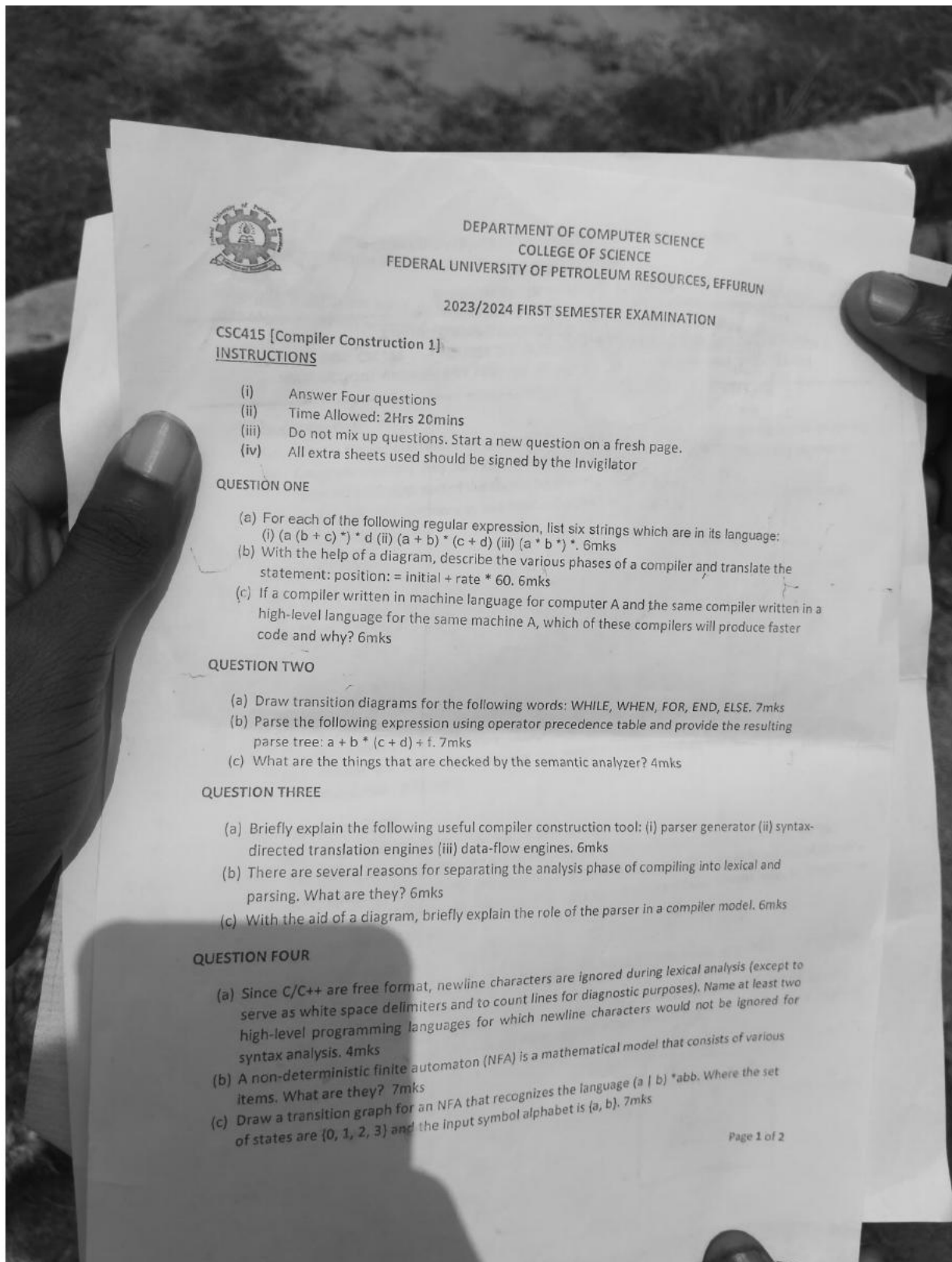
0 → 1 → 2 → 3

0 → 1 → 2 → 3

Paper 4 of 6 — CSC 415 PAST QUESTION

Level 400 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



QUESTION FIVE

- (a) Briefly explain the various error-recovery strategies that a parser can employ to recover from a syntactic error. 6mks
- (b) Determine whether the following grammar is ambiguous. If so, show two different derivation trees for the same string of terminals, and shows a left-most derivation corresponding to each tree. 6mks

1. $S \rightarrow aSbS$
2. $S \rightarrow As$
3. $S \rightarrow c$

- (c) Given a grammar: 1: $S \rightarrow ASB$. 6mks

- 2: $S \rightarrow \epsilon$
- 3: $A \rightarrow a$
- 4: $B \rightarrow b$

Draw a derivation tree for aaabbb



FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

COLLEGE OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

FIRST SEMESTER EXAMINATION 2022/2023 SESSION

COURSE CODE: CSC415

COURSE TITLE: COMPILER CONSTRUCTION 1

DURATION: 2HRS 15MINS

INSTRUCTION: ANSWER ANY FOUR QUESTIONS ONLY

QUESTION ONE

- What type of errors the lexical analyzer report? Support your answer with example. 4mks
- What are the different phases of compiler? Explain the phases in details. Write down the output of each phase for the expression $a := b + c * 50$. 8mks
- If a compiler is written in machine language for computer A, and the same compiler written in a high language for the same machine A, which of these compilers will produce faster code and why? 6mks

QUESTION TWO

- Briefly explain the issues associated with lexical analysis. 4mks
- Briefly explain the concept of finite automata and indicate what a nondeterministic finite automata consists of. 8mks
- Draw a syntax tree for the expression $a * (b + c) / 4$. 6mks

QUESTION THREE

- Define a context-free grammar and briefly explain the various Chomsky's classification of grammars. 7mks
- What is the difference between source code, an intermediate code and an object code? 5mks
- Show three different derivations using the grammar shown below: 6mks
 - $S \rightarrow aSA$
 - $S \rightarrow BA$
 - $A \rightarrow ab$
 - $B \rightarrow bA$

QUESTION FOUR

- List and briefly explain the various compiler construction tools we have. 4mks
- Differentiate between tokens, patterns and lexeme. 6mks
- Draw a transition diagram for the following words: BEGIN, END, ELSE, IF and THEN. 8mks

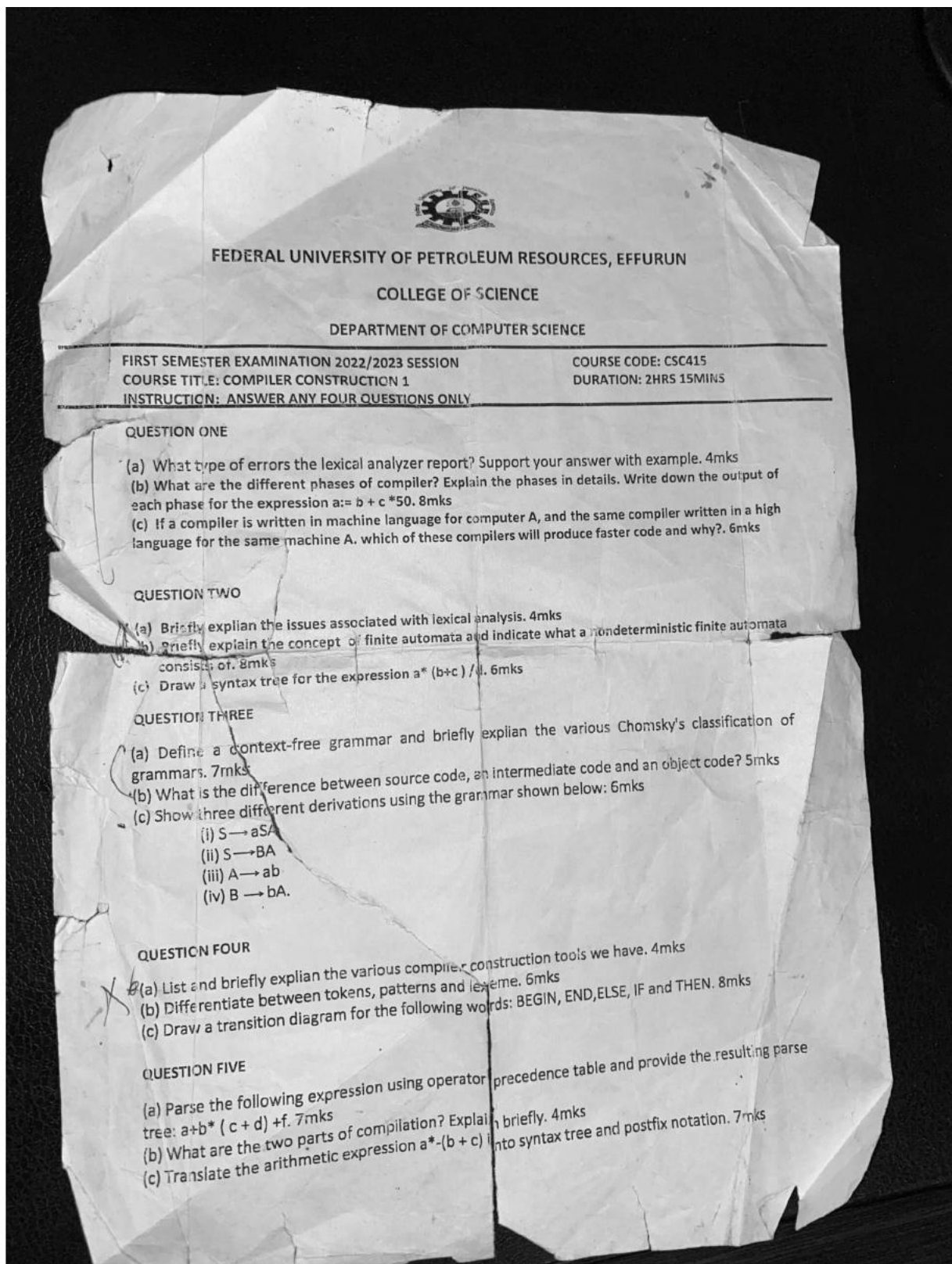
QUESTION FIVE

- Parse the following expression using operator precedence table and provide the resulting parse tree: $a + b * (c + d) + f$. 7mks
- What are the two parts of compilation? Explain briefly. 4mks
- Translate the arithmetic expression $a * -(b + c)$ into syntax tree and postfix notation. 7mks

Paper 5 of 6 — CSC 415 PQ

Level 400 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.





DEPARTMENT OF COMPUTER SCIENCE
COLLEGE OF SCIENCE
FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

2023/2024 FIRST SEMESTER EXAMINATION

CSC415 [Compiler Construction 1]

INSTRUCTIONS

- (i) Answer Four questions
- (ii) Time Allowed: 2Hrs 20mins
- (iii) Do not mix up questions. Start a new question on a fresh page.
- (iv) All extra sheets used should be signed by the Invigilator

QUESTION ONE

- (a) For each of the following regular expression, list six strings which are in its language:
(i) $(a(b+c)^*)^*d$ (ii) $(a+b)^*(c+d)$ (iii) $(a^*b^*)^*$. 6mks
- (b) With the help of a diagram, describe the various phases of a compiler and translate the statement: $\text{position} = \text{initial} + \text{rate} * 60$. 6mks
- (c) If a compiler written in machine language for computer A and the same compiler written in a high-level language for the same machine A, which of these compilers will produce faster code and why? 6mks

QUESTION TWO

- (a) Draw transition diagrams for the following words: WHILE, WHEN, FOR, END, ELSE. 7mks
- (b) Parse the following expression using operator precedence table and provide the resulting parse tree: $a + b * (c + d) + f$. 7mks
- (c) What are the things that are checked by the semantic analyzer? 4mks

QUESTION THREE

- (a) Briefly explain the following useful compiler construction tool: (i) parser generator (ii) syntax-directed translation engines (iii) data-flow engines. 6mks
- (b) There are several reasons for separating the analysis phase of compiling into lexical and parsing. What are they? 6mks
- (c) With the aid of a diagram, briefly explain the role of the parser in a compiler model. 6mks

QUESTION FOUR

- (a) Since C/C++ are free format, newline characters are ignored during lexical analysis (except to serve as white space delimiters and to count lines for diagnostic purposes). Name at least two high-level programming languages for which newline characters would not be ignored for syntax analysis. 4mks
- (b) A non-deterministic finite automaton (NFA) is a mathematical model that consists of various items. What are they? 7mks
- (c) Draw a transition graph for an NFA that recognizes the language $(a|b)^*abb$. Where the set of states are $\{0, 1, 2, 3\}$ and the input symbol alphabet is $\{a, b\}$. 7mks

Page 1 of 2



DEPARTMENT OF COMPUTER SCIENCE
COLLEGE OF SCIENCE
FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

2023/2024 FIRST SEMESTER EXAMINATION

CSC415 [Compiler Construction 1]

INSTRUCTIONS

- Answer Four questions
- Time Allowed: 2Hrs 20mins
- Do not mix up questions. Start a new question on a fresh page.
- All extra sheets used should be signed by the Invigilator

QUESTION ONE

- For each of the following regular expression, list six strings which are in its language:
(i) $(a(b+c)^*)^*d$ (ii) $(a+b)^*(c+d)$ (iii) $(a^*b^*)^*$. 6mks
- With the help of a diagram, describe the various phases of a compiler and translate the statement: position = initial + rate * 60. 6mks
- If a compiler written in machine language for computer A and the same compiler written in a high-level language for the same machine A, which of these compilers will produce faster code and why? 6mks

QUESTION TWO

- Draw transition diagram for the following words: WHILE, WHEN, FOR, END, ELSE. 7mks
- Parse the following expression using operator precedence table and provide the resulting parse tree: $a + b * (c + d) + f$. 7mks
- What are the things that are checked by the semantic analyzer? 4mks

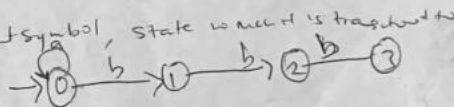
QUESTION THREE

- Briefly explain the following useful compiler construction tool: (i) parser generator (ii) syntax-directed translation engines (iii) data-flow engines. 6mks
- There are several reasons for separating the analysis phase of compiling into lexical and parsing. What are they? 6mks
- With the aid of a diagram, briefly explain the role of the parser in a compiler model. 6mks

QUESTION FOUR

- Since C/C++ are free format, newline characters are ignored during lexical analysis (except to serve as white space delimiters and to count lines for diagnostic purposes). Name at least two high-level programming languages for which newline characters would not be ignored for syntax analysis. 4mks
- A non-deterministic finite automaton (NFA) is a mathematical model that consists of various items. What are they? 7mks
- Draw a transition graph for an NFA that recognizes the language $(a|b)^*abb$. Where the set of states are $\{0, 1, 2, 3\}$ and the input symbol alphabet is $\{a, b\}$. 7mks

Can a state have more than one loop
a set of states designated as accepting
designating as accepting
a transition function / input symbol



Page 1 of 2

QUESTION FIVE

- (a) Briefly explain the various error-recovery strategies that a parser can employ to recover from a syntactic error. 6mks
- (b) Determine whether the following grammar is ambiguous. If so, show two different derivation trees for the same string of terminals, and shows a left-most derivation corresponding to each tree. 6mks
1. $S \rightarrow aSbS$
 2. $S \rightarrow As$
 3. $S \rightarrow c$
- (c) Given a grammar: 1: $S \rightarrow ASB$. 6mks
- 2: $S \rightarrow \epsilon$
 - 3: $A \rightarrow a$
 - 4: $B \rightarrow b$

Draw a derivation tree for aaabbb

Paper 6 of 6 — CSC 415 handout

Level 400 • First Semester • Handout

COMPILER CONSTRUCTION

COMPILER

INTRODUCTION

A diagram of the operation of a typical multi-language, multi-target compiler

A compiler is a computer program (or set of programs) that transforms source code written in a programming language (the source language) into another computer language (the target language, often having a binary form known as object code). The

COMPILED BY ABERE REUBEN Page 1

most common reason for wanting to transform source code is to create an executable program.

The name "compiler" is primarily used for programs that translate source code from a high-level programming language to a lower level language (e.g., assembly language or machine code). If the compiled program can run on a computer whose CPU or operating system is different from the one on which the compiler runs, the compiler is known as a cross-compiler. A program that translates from a low level language to a higher level one is a decompiler. A program that translates between high-level languages is usually called a language translator, source to source translator, or language converter. A language rewriter is usually a program that translates the form of expressions without a change of language.

A compiler is likely to perform many or all of the following operations: lexical analysis, preprocessing, parsing, semantic analysis (Syntax-directed translation), code generation, and code optimization.

Program faults caused by incorrect compiler behavior can be very difficult to track down and work around; therefore, compiler implementors invest significant effort to ensure the correctness of their software.

Structure of a compiler

Compilers bridge source programs in high-level languages with the underlying hardware. A compiler requires 1) determining the correctness of the syntax of programs, 2) generating correct and efficient object code, 3) run-time organization, and 4) formatting output according to assembler and/or linker conventions. A compiler consists of three main parts: the frontend, the middle-end, and the backend.

The front end checks whether the program is correctly written in terms of the programming language syntax and semantics. Here legal and illegal programs are recognized. Errors are reported, if any, in a useful way. Type checking is also performed by collecting type information. The frontend then generates an intermediate representation or IR of the source code for processing by the middle-end.

The middle end is where optimization takes place. Typical transformations for optimization are removal of useless or unreachable code, discovery and propagation of constant values, relocation of computation to a less frequently executed place (e.g., out of a loop), or specialization of computation based on the context. The middle-end generates another IR for the following backend. Most optimization efforts are focused on this part.

COMPILED BY ABERE REUBEN Page 2

The back end is responsible for translating the IR from the middle-end into assembly code. The target instruction(s) are chosen for each IR instruction. Register allocation assigns processor registers for the program variables where possible. The backend utilizes the hardware by figuring out how to keep parallel execution units busy, filling delay slots, and so on. Although most algorithms for optimization are in NP, heuristic techniques are well-developed.

Compiler output

One classification of compilers is by the platform on which their generated code executes. This is known as the target platform.

A native or hosted compiler is one which output is intended to directly run on the same type of computer and operating system that the compiler itself runs on. The output of a cross compiler is designed to run on a different platform. Cross compilers are often used when developing software for embedded systems that are not intended to support a software development environment.

The output of a compiler that produces code for a virtual machine (VM) may or may not be executed on the same platform as the compiler that produced it. For this reason such compilers are not usually classified as native or cross compilers.

The lower level language that is the target of a compiler may itself be a high-level programming language. C, often viewed as some sort of portable assembler, can also be the target language of a compiler. E.g.: Cfront, the original compiler for C++ used C as target language. The C created by such a compiler is usually not intended to be read and maintained by humans. So indent style and pretty C intermediate code are irrelevant. Some features of C turn it into a good target language. E.g.: C code with #line directives can be generated to support debugging of the original source.

Compiled versus interpreted languages

Higher-level programming languages usually appear with a type of translation in mind: either designed as compiled language or interpreted language. However, in practice there is rarely anything about a language that requires it to be exclusively compiled or exclusively interpreted, although it is possible to design languages that rely on re-interpretation at run time. The categorization usually reflects the most popular or widespread implementations of a language — for instance, BASIC is sometimes called an interpreted language, and C a compiled one, despite the existence

of BASIC compilers and C interpreters.

COMPILED BY ABERE REUBEN Page 3

Interpretation does not replace compilation completely. It only hides it from the user and makes it gradual. Even though an interpreter can itself be interpreted, a directly executed program is needed somewhere at the bottom of the stack. Modern trends toward just-in-time compilation and bytecode interpretation at times blur the traditional categorizations of compilers and interpreters.

Some language specifications spell out that implementations must include a compilation facility; for example, Common Lisp. However, there is nothing inherent in the definition of Common Lisp that stops it from being interpreted. Other languages have features that are very easy to implement in an interpreter, but make writing a compiler much harder; for example, APL, SNOBOL4, and many scripting languages allow programs to construct arbitrary source code at runtime with regular string operations, and then execute that code by passing it to a special evaluation function. To implement these features in a compiled language, programs must usually be shipped with a runtime library that includes a version of the compiler itself.

Compiler construction

In the early days, the approach taken to compiler design used to be directly affected by the complexity of the processing, the experience of the person(s) designing it, and the resources available.

A compiler for a relatively simple language written by one person might be a single, monolithic piece of software. When the source language is large and complex, and high quality output is required, the design may be split into a number of relatively independent phases. Having separate phases means development can be parceled up into small parts and given to different people. It also becomes much easier to replace a single phase by an improved one or to insert new phases later (e.g., additional optimizations).

The division of the compilation processes into phases was championed by the Production Quality Compiler-Compiler Project (PQCC) at Carnegie Mellon University. This project introduced the terms front end, middle end, and back end.

All but the smallest of compilers have more than two phases. However, these phases are usually regarded as being part of the front end or the back end. The point at which these two ends meet is open to debate. The front end is generally considered to be where syntactic and semantic processing takes place, along with translation to a lower level of representation (than source code).

COMPILED BY ABERE REUBEN Page 4

The middle end is usually designed to perform optimizations on a form other than the source code or machine code. This source code/machine code independence is

intended to enable generic optimizations to be shared between versions of the compiler supporting different languages and target processors.

The back end takes the output from the middle. It may perform more analysis, transformations and optimizations that are for a particular computer. Then, it generates code for a particular processor and OS.

This front-end/middle/back-end approach makes it possible to combine front ends for different languages with back ends for different CPUs. Practical examples of this approach are the GNU Compiler Collection, LLVM, and the Amsterdam Compiler Kit, which have multiple front-ends, shared analysis and multiple back-ends.

One-pass versus multi-pass compilers

Classifying compilers by number of passes has its background in the hardware resource limitations of computers. Compiling involves performing lots of work and early computers did not have enough memory to contain one program that did all of this work. So compilers were split up into smaller programs which each made a pass over the source (or some representation of it) performing some of the required analysis and translations.

The ability to compile in a single pass has classically been seen as a benefit because it simplifies the job of writing a compiler and one-pass compilers generally perform compilations faster than multi-pass compilers. Thus, partly driven by the resource limitations of early systems, many early languages were specifically designed so that they could be compiled in a single pass (e.g., Pascal).

In some cases the design of a language feature may require a compiler to perform more than one pass over the source. For instance, consider a declaration appearing on line 20 of the source which affects the translation of a statement appearing on line 10. In this case, the first pass needs to gather information about declarations appearing after statements that they affect, with the actual translation happening during a subsequent pass.

The disadvantage of compiling in a single pass is that it is not possible to perform many of the sophisticated optimizations needed to generate high quality code. It can be difficult to count exactly how many passes an optimizing compiler makes. For instance, different phases of optimization may analyse one expression many times but only analyse another expression once.

COMPILED BY ABERE REUBEN Page 5

Splitting a compiler up into small programs is a technique used by researchers interested in producing provably correct compilers. Proving the correctness of a set of small programs often requires less effort than proving the correctness of a larger, single, equivalent program.

While the typical multi-pass compiler outputs machine code from its final pass, there are several other types:

- A "source-to-source compiler" is a type of compiler that takes a high level language as its input and outputs a high level language. For example, an automatic parallelizing compiler will frequently take in a high level language program as an input and then transform the code and annotate it with parallel code annotations (e.g. OpenMP) or language constructs (e.g. Fortran's DOALL statements).
- Stage compiler that compiles to assembly language of a theoretical machine, like some Prolog implementations
 - o This Prolog machine is also known as the Warren Abstract Machine (or WAM).
 - o Bytecode compilers for Java, Python, and many more are also a subtype of this.
- Just-in-time compiler, used by Smalltalk and Java systems, and also by Microsoft .NET's Common Intermediate Language (CIL)
 - o Applications are delivered in bytecode, which is compiled to native machine code just prior to execution.

Front end

The compiler frontend analyzes the source code to build an internal representation of the program, called the intermediate representation or IR. It also manages the symbol table, a data structure mapping each symbol in the source code to associated information such as location, type and scope. This is done over several phases, which includes some of the following:

1. Line reconstruction. Languages which stop their keywords or allow arbitrary spaces within identifiers require a phase before parsing, which converts the input character sequence to a canonical form ready for the parser. The top-down, recursive-descent, table-driven parsers used in the 1960s typically read the source one character at a time and did not require a separate tokenizing phase. Atlas Autocode, and Imp (and some implementations of ALGOL and Coral 66) are examples of stopped languages which compilers would have a Line Reconstruction phase.

COMPILED BY ABERE REUBEN Page 6

2. Lexical analysis breaks the source code text into small pieces called tokens. Each token is a single atomic unit of the language, for instance a keyword, identifier or symbol name. The token syntax is typically a regular language, so a finite state automaton constructed from a regular expression can be used to recognize it. This phase is also called lexing or scanning, and the software doing lexical analysis is called a lexical analyzer or scanner.
3. Preprocessing. Some languages, e.g., C, require a preprocessing phase which supports macro substitution and conditional compilation. Typically the preprocessing phase occurs before syntactic or semantic analysis; e.g. in the

case of C, the preprocessor manipulates lexical tokens rather than syntactic forms. However, some languages such as Scheme support macro substitutions based on syntactic forms.

4. Syntax analysis involves parsing the token sequence to identify the syntactic structure of the program. This phase typically builds a parse tree, which replaces the linear sequence of tokens with a tree structure built according to the rules of a formal grammar which define the language's syntax. The parse tree is often analyzed, augmented, and transformed by later phases in the compiler.

5. Semantic analysis is the phase in which the compiler adds semantic information to the parse tree and builds the symbol table. This phase performs semantic checks such as type checking (checking for type errors), or object binding (associating variable and function references with their definitions), or definite assignment (requiring all local variables to be initialized before use), rejecting incorrect programs or issuing warnings. Semantic analysis usually requires a complete parse tree, meaning that this phase logically follows the parsing phase, and logically precedes the code generation phase, though it is often possible to fold multiple phases into one pass over the code in a compiler implementation.

Back end

The term back end is sometimes confused with code generator because of the overlapped functionality of generating assembly code. Some literature uses middle end to distinguish the generic analysis and optimization phases in the back end from the machine-dependent code generators.

The main phases of the back end include the following:

1. Analysis: This is the gathering of program information from the intermediate representation derived from the input. Typical analyses are data flow analysis to build use-define chains, dependence analysis, alias analysis, pointer analysis, escape analysis etc. Accurate analysis is the basis for any compiler
COMPILED BY ABERE REUBEN Page 7

optimization. The call graph and control flow graph are usually also built during the analysis phase.

2. Optimization: the intermediate language representation is transformed into functionally equivalent but faster (or smaller) forms. Popular optimizations are inline expansion, dead code elimination, constant propagation, loop transformation, register allocation and even automatic parallelization.

3. Code generation: the transformed intermediate language is translated into the output language, usually the native machine language of the system. This involves resource and storage decisions, such as deciding which variables to fit into registers and memory and the selection and scheduling of appropriate

machine instructions along with their associated addressing modes. Debug data may also need to be generated to facilitate debugging.

Compiler analysis is the prerequisite for any compiler optimization, and they tightly work together. For example, dependence analysis is crucial for loop transformation.

In addition, the scope of compiler analysis and optimizations vary greatly, from as small as a basic block to the procedure/function level, or even over the whole program (inter-procedural optimization). Obviously, a compiler can potentially do a better job using a broader view. But that broad view is not free: large scope analysis and optimizations are very costly in terms of compilation time and memory space; this is especially true for inter-procedural analysis and optimizations.

Inter-procedural analysis and optimizations are common in modern commercial compilers from HP, IBM, SGI, Intel, Microsoft, and Sun Microsystems. The open source GCC was criticized for a long time for lacking powerful inter-procedural optimizations, but it is changing in this respect. Another open source compiler with full analysis and optimization infrastructure is Open64, which is used by many organizations for research and commercial purposes.

Due to the extra time and space needed for compiler analysis and optimizations, some compilers skip them by default. Users have to use compilation options to explicitly tell the compiler which optimizations should be enabled.

Compiler correctness

Compiler correctness is the branch of software engineering that deals with trying to show that a compiler behaves according to its language specification. Techniques include developing the compiler using formal methods and using rigorous testing (often called compiler validation) on an existing compiler.

COMPILED BY ABERE REUBEN Page 8

GRAMMAR AND LANGUAGE

What is a grammar?

A grammar is a powerful tool for describing and analyzing languages. It is a set of rules by which valid sentences in a language are constructed. Here's a trivial example of English grammar:

sentence -> <subject> <verb-phrase> <object>

subject -> This | Computers | I

verb-phrase -> <adverb> <verb> | <verb>

adverb -> never

verb -> is | run | am | tell

object -> the <noun> | a <noun> | <noun>

noun -> university | world | cheese | lies

Using the above rules or productions, we can derive simple sentences such as these:

This is a university.

Computers run the world.

I am the cheese.

I never tell lies.

Here is a leftmost derivation of the first sentence using these productions.

Sentence $\rightarrow$ <subject> <verb-phrase> <object>

$\rightarrow$ This <verb-phrase> <object>

$\rightarrow$ This <verb> <object>

$\rightarrow$ This is <object>

$\rightarrow$ This is a <noun>

$\rightarrow$ This is a university

In addition to several reasonable sentences, we can also derive nonsense like

"Computers run cheese" and "This am a lies". These sentences don't make semantic sense, but they are syntactically correct because they are of the sequence of subject, verb-phrase, and object. Formal grammars are a tool for syntax, not semantics. We

COMPILED BY ABERE REUBEN Page 9

worry about semantics at a later point in the compiling process. In the syntax analysis phase, we verify structure, not meaning.

Vocabulary

We need to review some definitions before we can proceed:

grammar a set of rules by which valid sentences in a language are constructed.

nonterminal a grammar symbol that can be replaced/expanded to a sequence of symbols.

terminal an actual word in a language; these are the symbols in a grammar that cannot be replaced by anything else. "terminal" is supposed to conjure up the idea that it is a dead-end—no further expansion is possible.

production a grammar rule that describes how to replace/exchange symbols. The general form of a production for a nonterminal is:

$X \rightarrow Y_1Y_2Y_3\dots Y_n$

The nonterminal X is declared equivalent to the concatenation of the symbols $Y_1Y_2Y_3\dots Y_n$. The production means that anywhere where we encounter X , we may replace it by the string $Y_1Y_2Y_3\dots Y_n$. Eventually we will have a string containing nothing that can be expanded further, i.e., it will consist of only terminals. Such a string is called a sentence. In the context of programming languages, a sentence is a syntactically correct and complete program

derivation a sequence of applications of the rules of a grammar that produces a finished string of terminals. A leftmost derivation is where we always substitute for the leftmost nonterminal as we apply the rules (we can similarly define a rightmost derivation). A derivation is also called a parse.

start symbol a grammar has a single nonterminal (the start symbol) from which all sentences derive:

$S \rightarrow X_1X_2X_3\dots X_n$

COMPILED BY ABERE REUBEN Page 10

All sentences are derived from S by successive replacement using the productions of the grammar.

null symbol ϵ it is sometimes useful to specify that a symbol can be replaced by nothing at all. To indicate this, we use the null symbol ϵ , e.g., $A \rightarrow B \mid \epsilon$.

BNF a way of specifying programming languages using formal grammars and production rules with a particular form of notation (Backus-Naur form).

Parse Representation

In working with grammars, we can represent the application of the rules to derive a sentence in two ways. The first is a derivation as shown earlier for "This is a university" where the rules are applied step-by-step and we substitute for one nonterminal at a time. Think of a derivation as a history of how the sentence was parsed because it not only includes which productions were applied, but also the order they were applied (i.e., which nonterminal was chosen for expansion at each step). There can many different derivations for the same sentence (the leftmost, the rightmost, and so on).

A parse tree is the second method for representation. It diagrams how each symbol derives from other symbols in a hierarchical manner. Here is a parse tree for "This is a university":

S

Subject V-P object

This verb a noun

is university

COMPILED BY ABERE REUBEN Page 11

Although the parse tree includes all of the productions that were applied, it does not encode the order they were applied. For an unambiguous grammar, there is exactly one parse tree for a particular sentence.

More Definitions

Here are some other definitions we will need, described in reference to this example grammar:

$S \rightarrow AB$

$A \rightarrow Ax \mid y$

$B \rightarrow z$

alphabet

The alphabet is $\{S, A, B, x, y, z\}$. It is divided into two disjoint sets. The terminal alphabet consists of terminals, which appear in the sentences of the language: $\{x, y, z\}$. The remaining symbols are the nonterminal alphabet; these are the symbols that appear on the left side of productions and can be replaced during the course of a

derivation: $\{S, A, B\}$. Formally, we use V for the alphabet, T for the terminal alphabet and N for the nonterminal alphabet giving us: $V = T \cup N$, and $T \cap N = \emptyset$.

The convention used in our lecture notes are a sans-serif font for grammar elements, lowercase for terminals, uppercase for nonterminals, and underlined lowercase (e.g., u , v) to denote arbitrary strings of terminal and nonterminal symbols (possibly null).

In some textbooks, Greek letters are used for arbitrary strings of terminal and nonterminal symbols (e.g., α , β)

context-free grammar

To define a language, we need a set of productions, of the general form: $u \rightarrow v$. In a context-free grammar, u is a single nonterminal and v is an arbitrary string of terminal and nonterminal symbols. When parsing, we can replace u by v wherever it occurs.

We shall refer to this set of productions symbolically as P .

COMPILED BY ABERE REUBEN Page 12

formal grammar

We formally define a grammar as a 4-tuple $\{S, P, N, T\}$. S is the start symbol and $S \in N$, P is the set of productions, and N and T are the nonterminal and terminal alphabets. A sentence is a string of symbols in T derived from S using one or more applications of productions in P . A string of symbols derived from S but possibly including nonterminals is called a sentential form or a working string.

A production $u \rightarrow v$ is used to replace an occurrence of u by v . Formally, if we apply a production $p \in P$ to a string of symbols w in V to yield a new string of symbols z in V , we say that z derived from w using p , written as follows: $w \Rightarrow_p z$.

We also use:

$w \Rightarrow z$ z derives from w (production unspecified)

$w \Rightarrow^* z$ z derives from w using zero or more productions

$w \Rightarrow^+ z$ z derives from w using one or more productions

equivalence

The language $L(G)$ defined by grammar G is the set of sentences derivable using G .

Two grammars G and G' are said to be equivalent if the languages they generate, $L(G)$ and $L(G')$, are the same.

Grammar Hierarchy

We owe a lot of our understanding of grammars to the work of the American linguist Noam Chomsky. There are four categories of formal grammars in the Chomsky Hierarchy, they span from Type 0, the most general, to Type 3, the most restrictive. More restrictions on the grammar make it easier to describe and efficiently parse, but reduce the expressive power.

Type 0: free or unrestricted grammars

These are the most general. Productions are of the form $u \rightarrow v$ where both u and v are arbitrary strings of symbols in V , with u non-null. There are no restrictions on what appears on the left or right-hand side other than the lefthand side must be non-empty.

COMPILED BY ABERE REUBEN Page 13

Type 1: context-sensitive grammars

Productions are of the form $uXw \rightarrow uvw$ where u , v and w are arbitrary strings of symbols in V , with v non-null, and X a single nonterminal. In other words, X may be replaced by v but only when it is surrounded by u and w . (i.e., in a particular context).

Type 2: context-free grammars

Productions are of the form $X \rightarrow v$ where v is an arbitrary string of symbols in V , and X is a single nonterminal. Wherever you find X , you can replace with v (regardless of context).

Type 3: regular grammars

Productions are of the form $X \rightarrow a$, $X \rightarrow aY$, or $X \rightarrow \epsilon$ where X and Y are nonterminals and a is a terminal. That is, the left-hand side must be a single nonterminal and the right-hand side can be either empty, a single terminal by itself or with a single nonterminal. These grammars are the most limited in terms of expressive power.

Every type 3 grammar is a type 2 grammar, and every type 2 is a type 1 and so on.

Type 3 grammars are particularly easy to parse because of the lack of recursive constructs. Efficient parsers exist for many classes of Type 2 grammars. Although Type 1 and Type 0 grammars are more powerful than Type 2 and 3, they are far less useful since we cannot create efficient parsers for them. In designing programming languages using formal grammars, we will use Type 2 or context-free grammars, often just abbreviated as CFG.

Issues in parsing context-free grammars

There are several efficient approaches to parsing most Type 2 grammars and we will talk through them over the next few lectures. However, there are issues that can interfere with parsing that we must take into consideration when designing the grammar. Let's take a look at three of them: ambiguity, recursive rules, and left-factoring.

COMPILED BY ABERE REUBEN Page 14

Ambiguity

If a grammar permits more than one parse tree for some sentences, it is said to be ambiguous. For example, consider the following classic arithmetic expression grammar:

$E \rightarrow E \text{ op } E \mid (E) \mid \text{int}$

$\text{Op} \rightarrow + \mid - \mid * \mid /$

This grammar denotes expressions that consist of integers joined by binary operators and possibly including parentheses. As defined above, this grammar is ambiguous because for certain sentences we can construct more than one parse tree. For example, consider the expression $10 - 2 * 5$. We parse by first applying the production $E \rightarrow E \text{ op } E$. The parse tree on the left chooses to expand that first op to $*$, the one on the

right to -. We have two completely different parse trees. Which one is correct?

E

E OP E

E OP E * int

int - int 5

10 2

E

E OP E

int - E OP E

int * int

10 2 5

COMPILED BY ABERE REUBEN Page 15

Both trees are legal in the grammar as stated and thus either interpretation is valid. Although natural languages can tolerate some kind of ambiguity (e.g., puns, plays on words, etc.), it is not acceptable in computer languages. We don't want the compiler just haphazardly deciding which way to interpret our expressions! Given our expectations from algebra concerning precedence, only one of the trees seems right. The right-hand tree fits our expectation that * "binds tighter" and for that result to be computed first then integrated in the outer expression which has a lower precedence operator. It's fairly easy for a grammar to become ambiguous if you are not careful in its construction. Unfortunately, there is no magical technique that can be used to resolve all varieties of ambiguity. It is an undecidable problem to determine whether any grammar is ambiguous, much less to attempt to mechanically remove all ambiguity. However, that doesn't mean in practice that we cannot detect ambiguity or do something about it. For programming language grammars, we usually take pains to construct an unambiguous grammar or introduce additional disambiguating rules to throw away the undesirable parse trees, leaving only one for each sentence.

Using the above ambiguous expression grammar, one technique would leave the grammar as is, but add disambiguating rules into the parser implementation. We could code into the parser knowledge of precedence and associativity to break the tie and force the parser to build the tree on the right rather than the left. The advantage of this is that the grammar remains simple and less complicated. But as a downside, the syntactic structure of the language is no longer given by the grammar alone.

Another approach is to change the grammar to only allow the one tree that correctly reflects our intention and eliminate the others. For the expression grammar, we can separate expressions into multiplicative and additive subgroups and force them to be expanded in the desired order.

COMPILED BY ABERE REUBEN Page 16

E -> E t_{op} E | T

t_{op} -> +|-

$T \rightarrow T \text{ f_op } T \mid F$
 $\text{f_op} \rightarrow * \mid /$
 $F \rightarrow (E) \mid \text{int}$

Terms are addition/subtraction expressions and factors used for multiplication and division. Since the base case for expression is a term, addition and subtraction will appear higher in the parse tree, and thus receive lower precedence. After verifying that the above re-written grammar has only one parse tree for the earlier ambiguous expression, you might think we were home free, but now consider the expression $10 - 2 - 5$. The recursion on both sides of the binary operator allows either side to match repetitions. The arithmetic operators usually associate to the left, so by replacing the right-hand side with the base case will force the repetitive matches onto the left side. The final result is:

$E \rightarrow E \text{ t_op } T \mid T$
 $\text{t_op} \rightarrow + \mid -$
 $T \rightarrow T \text{ f_op } F \mid F$
 $\text{f_op} \rightarrow * \mid /$
 $F \rightarrow (E) \mid \text{int}$

Whew! The obvious disadvantage of changing the grammar to remove ambiguity is that it may complicate and obscure the original grammar definitions. There is no mechanical means to change any ambiguous grammar into an unambiguous one (undecidable, remember?) However, most programming languages have only limited issues with ambiguity that can be resolved using ad hoc techniques.

Recursive productions

Productions are often defined in terms of themselves. For example a list of variables in a programming language grammar could be specified by this production:

COMPILED BY ABERE REUBEN Page 17

$\text{variable_list} \rightarrow \text{variable} \mid \text{variable_list} , \text{variable}$

Such productions are said to be recursive. If the recursive nonterminal is at the left of the right-side of the production, e.g. $A \rightarrow u \mid Av$, we call the production left-recursive. Similarly, we can define a right-recursive production: $A \rightarrow u \mid vA$. Some parsing techniques have trouble with one or the other variants of recursive productions and so sometimes we have to massage the grammar into a different but equivalent form. Left-recursive productions can be especially troublesome in the top-down parsers. Handily, there is a simple technique for rewriting the grammar to move the recursion to the other side. For example, consider this left-recursive rule:

$X \rightarrow Xa \mid Xb \mid AB \mid C \mid DEF$

To convert the rule, we introduce a new nonterminal X' that we append to the end of all non-left-recursive productions for X . The expansion for the new nonterminal is basically the reverse of the original left-recursive rule. The re-written productions are:

$X \rightarrow ABX' \mid CX' \mid DEFX'$
 $X' \rightarrow aX' \mid bX' \mid \epsilon$

It appears we just exchanged the left-recursive rules for an equivalent right-recursive version. This might seem pointless, but some parsing algorithms prefer or even require only left or right recursion.

Left-factoring

The parser usually reads tokens from left to right and it is convenient if, upon reading a token, it can make an immediate decision about which production from the grammar to expand. However, this can be trouble if there are productions that have common first symbol(s) on the right side of the productions. Here is an example we often see in programming language grammars:

Stmt $\rightarrow$ if Cond then Stmt else Stmt | if Cond then Stmt | Other |

The common prefix is if Cond then Stmt. This causes problems because when a parser encounter an "if", it does not know which production to use. A useful technique called

left-factoring allows us to restructure the grammar to avoid this situation. We rewrite the productions to defer the decision about which of the options to choose until we have seen enough of the input to make the appropriate choice. We factor out the common part of the two options into a shared rule that both will use and then add a new rule that picks up where the tokens diverge.

Stmt $\rightarrow$ if Cond then Stmt OptElse | Other | ...

OptElse $\rightarrow$ else S | ϵ

In the re-written grammar, upon reading an "if" we expand first production and wait until if Cond then Stmt has been seen to decide whether to expand OptElse to else or ϵ .

Hidden left-factors and hidden left recursion

A grammar may not appear to have left recursion or left factors, yet still have issues that will interfere with parsing. This may be because the issues are hidden and need to be first exposed via substitution. For example, consider this grammar:

A $\rightarrow$ da | acB

B $\rightarrow$ abB | daA | Af

A cursory examination of the grammar may not detect that the first and second productions of B overlap with the third. We substitute the expansions for A into the third production to expose this:

A $\rightarrow$ da | acB

B $\rightarrow$ abB | daA | daf | acBf

This exchanges the original third production of B for several new productions, one for each of the productions for A. These directly show the overlap, and we can then left factor:

COMPILED BY ABERE REUBEN Page 19

A $\rightarrow$ da | acB

B $\rightarrow$ aM | daN

$M \rightarrow bB \mid cBf$

$N \rightarrow A \mid f$

Similarly, the following grammar does not appear to have any left-recursion:

$S \rightarrow Tu \mid wx$

$T \rightarrow Sq \mid vvS$

Yet after substitution of S into T , the left-recursion comes to light:

$S \rightarrow Tu \mid wx$

$T \rightarrow Tuq \mid wxq \mid vvS$

If we then eliminate left-recursion, we get:

$S \rightarrow Tu \mid wx$

$T \rightarrow wxqT' \mid vvST'$

$T' \rightarrow uqT' \mid \epsilon$

COMPILED BY ABERE REUBEN Page 20